Le pouvoir, les pièges des pointeurs et la programmation défensive en C

David Guennec

C sécurisé II – CUPGE2 Cyber



Objectifs de ce module

Le présent cours vise à :

- renforcer les compétences acquises en programmation C lors du module de C sécurisé I
- vous donner des fondements de programmation défensive

Les sujets abordés seront les suivants :

- les pointeurs en C
- les principaux problèmes de programmation en C
- Programmation défensive quid est ?

* c'est quoi

La méthodologie sera en 2 temps :

- Un peu de CM pour acquérir les bases
- Un projet pour les approfondir par la pratique

Déroulé du module & évaluation

2 séances (4h) de CM/TP : aujourd'hui & la semaine prochaine

6 séances (12h) de projet et 2 parties.

2 intervenants en projet : Elvire Houssou & David Guennec

- 4 séances (i.e. 8h) sur la **première partie** du projet
  - début le 18/11
  - dernière séance le 9/12 avec rendu le même jour
- 2 séances (i.e. 4h) sur la **seconde partie**
  - première séance le 16/12
  - dernière séance le 9/1 avec rendu le même jour

Évaluation en 2 temps :

~65% de la note

~35% de la note

Q. : Quel est le sujet du projet ?

R. : **Tout vient à point à qui sait attendre.**

Au cours de ces 2 séances de CM, nous allons parler de...

1. Pointeur & Mémoire
2. Les Écueils des Pointeurs I : Fuites et Pointeurs Suspendus
3. Les Écueils des Pointeurs II : Vulnérabilités critiques
4. Programmation défensive en C

Petit Quiz de début de module

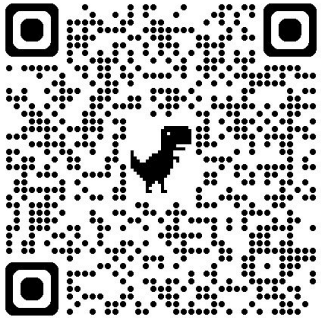
Dans ce cours, nous allons parler de **Programmation défensive**, à savoir comment produire un code le plus sûr possible.

Nous allons voir de nombreux concepts de base mais... peut-être en avez vous déjà certains.

Comment savoir si je fais déjà de la programmation défensive ?

Voici un petit quiz pour le découvrir→

(et soyez honnête avec vous-même dans les réponses, je ne peux pas les voir de toute façon).



Pointeurs & mémoire

Les pointeurs 👉

Dans le programme suivant, l'accès aux variables se fait par l'intermédiaire de son nom, lequel aura au préalable été déclaré :

```
int a = 3; int b; b = a; ++b;
```

On utilise donc les noms de "a" et "b" pour accéder d'abord à l'emplacement de a en mémoire, puis le dupliquer le contenu à l'emplacement de b avant, enfin, d'accéder à nouveau à l'emplacement de b pour en incrémenter le contenu.

Cela simplifie l'utilisation des emplacements mémoire car on utilise un nom bien commode pour nous humains.

Cependant, il n'en reste pas moins vrai que :

- toute variable est avant tout un emplacement mémoire contigu de 1 ou plusieurs octets ;
- cet emplacement est nécessairement référencé par une "adresse mémoire" univoque.

Les pointeurs 👉

```
int a = 3;
int b;
a = b;
++b;
```



📝 **Nom de variable**
Un nom pratique et lisible pour les humains (ex: "a", "b")

📍 **Adresse mémoire**
L'emplacement réel en mémoire (ex: 0x7fff5fbff8ac)

💾 **Valeur stockée**
Le contenu de la variable à cet emplacement (ex: 3)

Les pointeurs 👉

L'adresse restant bien moins lisible qu'un nom de variable, on préfère souvent utiliser ce dernier pour accéder à un emplacement mémoire. C'est alors le compilateur qui se charge d'identifier l'adresse à partir du nom de la variable. Cela dit, accéder à une variable par son adresse est possible, et c'est ce que permet un pointeur.

Donc concrètement, un pointeur, qu'est-ce que c'est ?

Définition

Un pointeur est une constante ou une variable stockant l'adresse* en mémoire d'une variable.

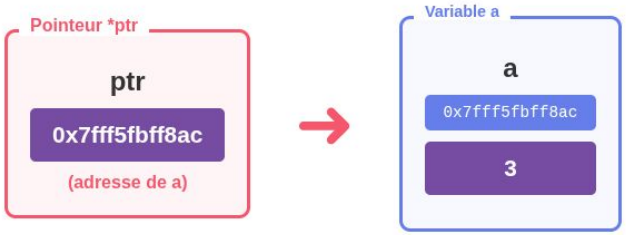
On dit qu'un pointeur "renvoie" ou "pointe" vers la variable concernée, dont il stocke l'adresse.

On dit aussi qu'un pointeur "fait référence" à la variable pointée.

Si c'est une variable, on peut mettre à jour l'adresse mémoire à volonté, comme pour toute variable.

* l'adresse du premier octet contenant la variable pointée.

Les pointeurs 👉




```c
int a = 3; // Variable normale
int *ptr = &a; // ptr stocke l'adresse de a
// ptr "pointe vers" a
```

Noter :

l'utilisation d'une étoile * dans la déclaration de la variable ptr.

l'utilisation d'une esperluette & (ou "et commercial") pour stocker l'adresse de a dans ptr.

***** est l'opérateur de **"déréférencement"** (déréférencement = accès à la donnée stockée).

& est l'opérateur **"d'adresse"** (accès à l'adresse mémoire d'une donnée).

Les mots clés &, * et NULL

Déclarer un pointeur :

type des valeurs pointées → `type * ptr;`

`type *` : c'est un pointeur vers la donnée en mémoire

`ptr` : nom de la variable

Accéder à la valeur pointée :

`*super_ptr` ← je veux accéder à l'emplacement mémoire contenu dans la variable "super_ptr"

Obtenir l'adresse d'une variable (n'importe laquelle) :

`&temp` ← je veux connaitre l'adresse de la variable "temp"

La valeur du pointeur est **NULL** ? Quésaco ?

La valeur **NULL** indique que le valeur ne pointe vers aucune valeur (absence d'adresse valide).

La définition de NULL → `#define NULL ((void*)0)`

Structures et pointeurs

Lorsque l'on utilise des **structures** (`struct`) et que cette dernière est accédée par un pointeur (ce qui est fréquent), on a une notation de type `(*var).champ`.

Cette notation peut être (et est quasiment toujours) abrégée en `var->champ`.

Exemple :

```c
// Définition de la structure Personne
struct Personne {
    char *nom;
    int age;
};

struct Personne *p = malloc(sizeof(struct Personne));
p->nom = "Alice"; // équivaut à (*p).nom = "Alice";
p->age = 30;      // équivaut à (*p).age = 30;
```

Avez-vous bien compris ?

Que veut dire la chose suivante :

```
super_ptr = &temp;
```

```
temp = *super_ptr;
```

Que fait le programme suivant :

```
void main() {
  int a = 24, b = 5;
  int *p1, *p2;
  p1 = &a;
  p2 = &b;
  *p1 = *p2;
}
```

Que se passe t-il si on fait : `p1 = p2;` à la place ?

Avez-vous bien compris ?

Que veut dire la chose suivante :

```
super_ptr = &temp;
```

```
temp = *super_ptr;
```

Que fait le programme suivant :

```
void main() {
  int a = 24, b = 5;
  int *p1, *p2;
  p1 = &a;
  p2 = &b;
  *p1 = *p2;
}
```

p1 contient l'adresse de a et p2 celle de b

Que se passe t-il si on fait : `p1 = p2;` à la place ?

p1 contient la même chose que p2 : l'adresse de b

Arithmétique des pointeurs

Un pointeur stocke l'adresse du premier octet contenant la variable pointée. Cette adresse est une valeur entière.

« L'arithmétique des pointeurs » est une **fonctionnalité puissante** qui se distingue de l'arithmétique des entiers.

Comparaison

Deux pointeurs sur le même type de données peuvent être comparés en usant des comparateurs usuels :

```
<, <= , >, >= , ==, !=
```

Arithmétique des pointeurs

Les opérations d'addition et soustraction sont autorisées sur les pointeurs, afin d'effectuer des pointages de valeurs relatifs à une adresse.

Addition et soustraction

Quand vous ajoutez 1 à un pointeur, vous ne lui ajoutez pas 1 octet. Vous le déplacez pour qu'il pointe sur **l'élément suivant** du même type.

Le compilateur connaît la taille du type pointé (`sizeof(type)`) et s'occupe du calcul :

- **`ptr + 1`** pour un `int*` avance de `1 * sizeof(int)` octets (souvent 4).
- **`ptr + 1`** pour un `char*` avance de `1 * sizeof(char)` octet (toujours 1).
- **`ptr + i`** avance de `i * sizeof(type)` octets.

Les soustractions permettent de revenir en arrière dans la mémoire.

Avant d'aller plus loin

Vérifions si vous avez tout compris jusqu'à présent avec ce nouveau quiz :



Stack versus Heap

Le langage C dispose de deux zones de mémoire principales pour stocker les données du programme :



La Pile (Stack)

Les variables locales des fonctions sont allouées sur la pile, dont la gestion est automatique et de type LIFO (dernier entré, premier sorti).



Le Tas (Heap)

le tas est une zone de mémoire de taille quasi-illimitée où le développeur doit allouer et libérer explicitement la mémoire à l'aide de fonctions comme **`malloc()`**, **`calloc()`** et **`free()`**.



La Stack (Pile)

Imaginez la Stack comme une pile d'assiettes : vous ajoutez (**push**) et retirez (**pop**) toujours par le haut. C'est une structure de type **LIFO** (Last-In, First-Out).

- **Allocation** : **Automatique**. La mémoire est allouée et libérée automatiquement par le compilateur lorsque vous entrez et quittez une fonction.
- **Contenu** : Stocke les **variables locales** (y compris les paramètres de fonction) et les adresses de retour des fonctions.
- **Vitesse** : Très **rapide**, car la gestion est simple et structurée (seulement un pointeur est déplacé).
- **Taille** : Généralement **petite** et **fixe** (définie au démarrage du programme ou du thread).
- Durée de vie : Les données existent seulement pendant l'exécution de la fonction où elles sont déclarées.



Le Heap (Tas)

Le Heap est une grande zone de mémoire non structurée. C'est un espace plus libre, mais qui nécessite une gestion **manuelle** par le programmeur.

- **Allocation** : **Dynamique**. Vous devez allouer la mémoire manuellement en utilisant des fonctions comme `malloc()`, `calloc()`, ou `realloc()`.
- **Contenu** : Utilisé pour stocker des données qui doivent survivre à la fin d'une fonction, comme des **structures de données complexes** (listes chaînées, arbres) ou des données de **grande** taille.
- **Vitesse** : Relativement **plus lent**, car le système doit chercher un bloc de mémoire libre disponible (ce qui peut fragmenter la mémoire).
- **Taille** : Généralement beaucoup **plus grande** que la Stack, limitée par la mémoire virtuelle du système d'exploitation.
- **Durée de vie** : Les données existent **jusqu'à ce que vous les libériez explicitement** (avec `free()`) ou que le programme se termine.

⚠️ **Attention** : Oublier de libérer la mémoire provoque des **fuites mémoire** (memory leaks) !

Stack versus Heap

Caractéristique	Stack (Pile)	Heap (Tas)
Allocation	Automatique par le compilateur	Manuelle par le programmeur (`malloc/free`)
Vitesse	Très rapide	Plus lent
Durée de vie	Limitée à la fonction (portée locale)	Jusqu'à libération explicite (portée globale potentielle)
Taille	Petite et fixe	Grande et variable
Erreurs courantes	Dépassement de pile (Stack Overflow)	Fuites de mémoire (Memory Leaks), double libération (Double Free), corruption de tas

Allocation dynamique avec malloc()

⚠️ L'utilisation de **malloc()** nécessite l'inclusion de **stdlib.h**

`malloc()` : Allocation de Mémoire Brute

malloc() est la fonction la plus courante. Elle alloue un bloc de mémoire d'une taille spécifiée en octets.

```
void* malloc( size_t size );
```

- **Rôle :** Alloue `size` octets sur le Heap.
- I**nitialisation :** Les octets alloués contiennent des **valeurs indéterminées (garbage values)**. Vous devez les initialiser vous-même.
- **Retour :** Un pointeur générique `void*` vers le début du bloc, ou `NULL` en cas d'échec d'allocation.

Exemple :

```
int *i_ptr = (int *) malloc( sizeof(int) ); // Alloue la taille d'un entier
```

Allocation dynamique avec calloc()

⚠️ L'utilisation de **calloc()** nécessite l'inclusion de **stdlib.h**

`calloc()` : Allocation et Initialisation (Zérotage)

Utilisée principalement pour allouer des tableaux, car elle prend le nombre d'éléments et leur taille.

```
void* calloc( size_t num, size_t size );
```

- **Rôle :** Alloue de la place pour `num` éléments de taille `size` (donc `num * size` octets au total).
- **Initialisation : Initialise tous les octets alloués à zéro (0)**. C'est la différence clé avec `malloc`.
- **Retour :** Un pointeur générique `void*` vers le début du bloc, ou `NULL` en cas d'échec.

Exemple :

```
int *arr = (int *) calloc( 5, sizeof(int) ); // Alloue 5 entiers et les met à 0
```

Allocation dynamique avec realloc()

⚠️ L'utilisation de **realloc()** nécessite l'inclusion de **stdlib.h**

`realloc()` : Modification de la Taille

Utilisée pour changer la taille d'un bloc de mémoire précédemment alloué par `malloc` ou `calloc`.

```
void* realloc( void* ptr, size_t new_size );
```

- **Rôle :** Redimensionne le bloc de mémoire pointé par `ptr` à `new_size` octets.
- **Comportement :**
  - Si l'espace est suffisant **juste après** le bloc actuel, `realloc` peut simplement étendre le bloc et retourner la même adresse.
  - Sinon, `realloc` alloue un **nouveau** bloc ailleurs, copie le contenu de l'ancien bloc dans le nouveau, puis libère l'ancien.
- **Retour :** La nouvelle adresse du bloc, qui peut être **différente** de `ptr`, ou `NULL` en cas d'échec.

Exemple :

```
arr = (int *) realloc( arr, 10 * sizeof(int) ); // Étend l'ancien tableau à 10 entiers
```

Libération de la mémoire avec free()

⚠️ L'utilisation de **free()** nécessite l'inclusion de **stdlib.h**

`free()` : Libération de la Mémoire

L'opération inverse et indispensable : rendre la mémoire au système pour qu'elle puisse être réutilisée.

```
void free( void* ptr );
```

- **Rôle :** Libère le bloc de mémoire du Heap qui commence à l'adresse `ptr`.
- **Pré-requis :** `ptr` doit avoir été retourné par un appel précédent à `malloc`, `calloc`, ou `realloc`.
- **Effet :** Le bloc est libéré, mais `ptr` devient un **pointeur suspendu (dangling pointer)**.

Bonne Pratique : Toujours suivre `free(ptr)` par `ptr = NULL;` pour éviter l'utilisation accidentelle du pointeur suspendu.

Allocation Statique vs. Dynamique

Allocation statique :

```c
#include <stdio.h>

void main() {

        const int array_size = 5;
        int array[array_size];

        // Initialization loop
        for (int i = 0; i < array_size; ++i) {
                array[i] = i*i;
        }

        printf("The array has been populated. We shall now display it
to the used (that is, you, dear reader :)\n"  );

        // Visualization loop
        for (int i = 0; i<array_size;i++) {
                printf("%d\n", array[i]);
        }

        printf("The array has been correctly displayed
(hopefully).\n" );

        // Due diligence
        // No free() is needed as the array is on the stack.
}
```

Allocation dynamique :

```c
#include <stdio.h>
#include <stdlib.h>

void main() {

        const int array_size = 5;
        int *array;
        array = (int *) malloc( array_size * sizeof( int) );

        // Initialization loop
        for (int i = 0; i < array_size; ++i) {
                *(array+i) = i*i;
        }

        printf("The array has been populated. We shall now display
it to the used (that is, you, dear reader :)\n"  );

        // Visualization loop
        for (int i = 0; i<array_size;i++) {
                printf("%d\n", *(array+i));
        }

        printf("The array has been correctly displayed
(hopefully).\n" );

        // Due diligence
        free(array);
        array=NULL;
}
```

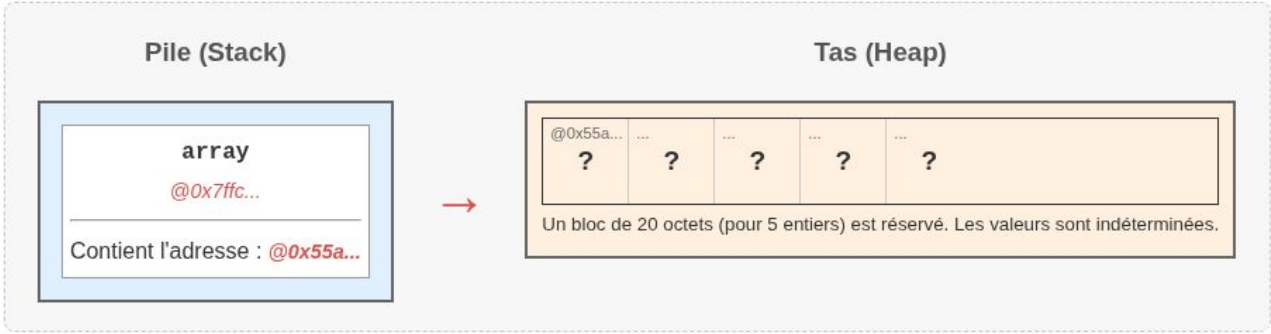
Allocation avec malloc

La première étape consiste à demander de la mémoire au système d'exploitation.

```
array = (int *) malloc( array_size * sizeof(int) );
```

- `malloc` (*memory allocation*) réserve un bloc de mémoire sur une zone appelée le **tas (heap)**.
- `array_size * sizeof(int)` calcule la taille totale nécessaire : 5 fois la taille d'un entier (par exemple, 5 * 4 = 20 octets).
- `malloc` retourne l'adresse du début de ce bloc.
- Cette adresse est stockée dans notre pointeur `array`, qui lui, se trouve sur la **pile (stack)**.

Visualisation de la mémoire :



Accès et Initialisation du tableau

Maintenant, nous allons remplir ce bloc mémoire en utilisant une boucle et l'**arithmétique des pointeurs**.

```
*(array + i) = i * i;
```

Décortiquons cette ligne cruciale :

- `array + i`: C'est le cœur de l'arithmétique des pointeurs. Cela ne signifie pas "adresse de base + i octets". Le compilateur sait que `array` est un pointeur vers des `int`. Donc, `array + i` signifie : "aller à l'adresse de base et avancer de `i` cases de la taille d'un `int`".
- `*( ... )`: L'opérateur de **déréférencement**. Il signifie : "accéder à la valeur qui se trouve à l'adresse calculée entre les parenthèses".

Accès et Initialisation du tableau

```
for (int i = 0; i < array_size; ++i) {
      *(array+i) = i*i;
}
```

Quand i = 0: `*(array + 0)` pointe vers le premier élément. On y écrit `0 * 0` :

0	**?**	**?**	**?**	**?**

Quand i = 1: `*(array + 1)` pointe vers le 1er élément + 1 (int) c'est à dire le deuxième élément. On y écrit `1 * 1` :

0	**1**	**?**	**?**	**?**

Quand i = 2: `*(array + 2)` pointe vers le 1er élément + 1 (int) c'est à dire le troisième élément. On y écrit `2 * 2` :

0	**1**	**4**	**?**	**?**

...et ainsi de suite jusqu'à la fin de la boucle.

État final après la boucle :

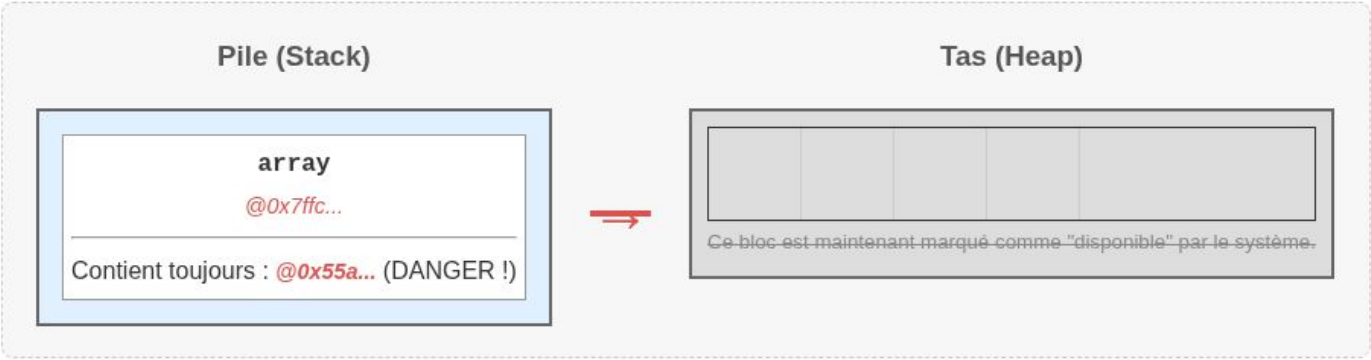
0	**1**	**4**	**9**	**16**

Libération de la mémoire

La mémoire allouée sur le tas ne se nettoie pas toute seule ! Nous devons la rendre explicitement au système.

```
free(array);
```

- `free()` prend en argument un pointeur qui a été retourné par `malloc`.
- Elle informe le système que le bloc de mémoire pointé par `array` est maintenant disponible et peut être réutilisé.
- **Attention :** Après `free(array)`, le pointeur `array` lui-même contient toujours l'ancienne adresse ! C'est ce qu'on appelle un **pointeur suspendu (dangling pointer)**. Tenter de l'utiliser est une grave erreur qui peut faire planter le programme.

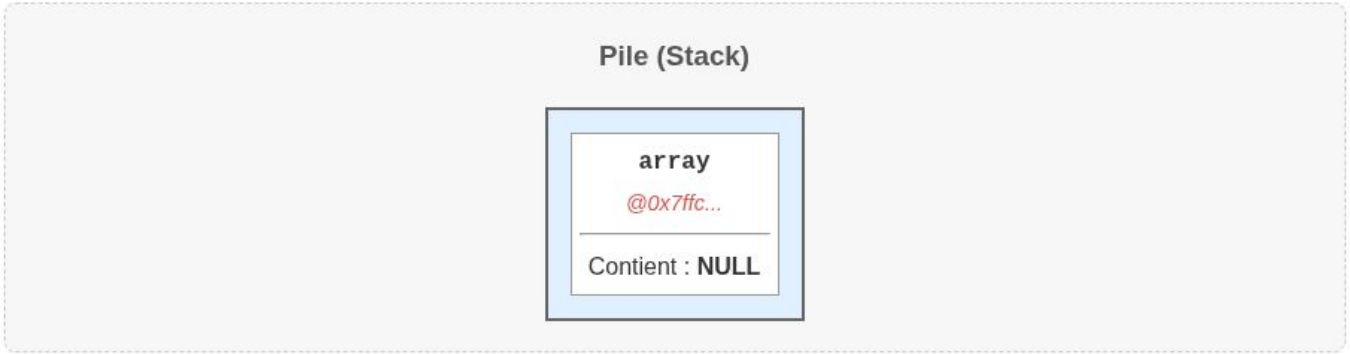


La Bonne Pratique : array = NULL;

Pour éviter les problèmes liés aux pointeurs suspendus, il est recommandé de mettre le pointeur à `NULL` juste après avoir libéré la mémoire.

```
array = NULL;
```

- `NULL` est une valeur spéciale (souvent 0) qui signifie "ce pointeur ne pointe sur rien de valide".
- Cela permet de vérifier plus tard si le pointeur est utilisable (ex: `if (array != NULL)`) et empêche son utilisation accidentelle.



Exercice

Code :

```c
#include <stdio.h>
#include <stdlib.h>

void main() {

        const int array_size = 5;
        int *array;
        array = (int *) malloc( array_size * sizeof( int) );

        // Initialization loop
        for (int i = 0; i < array_size; ++i) {
                *(array+i) = i*i;
        }

        printf("The array has been populated. We shall now display
it to the user (that is, you, dear reader :)\n"  );

        // Visualization loop
        for (int i = 0; i<array_size;i++) {
                printf("%d\n", *(array+i));
        }

        printf( "The array has been correctly displayed
(hopefully).\n" );

        // Due diligence
        free(array);
        array=NULL;
}
```

Reprendre le code à gauche de la manière suivante :

- au lieu de stocker des carrés, le tableau sera simplement initialisé avec la valeur 42 dans tous les emplacements.
- transformer les for en `while`.
- Ne pas utiliser de variable entière pour itérer dans les boucles (donc pas de variable comme `i` dans le code à gauche). Vous avez droit à une copie du pointeur `array` par contre.
- utiliser `array++` pour avancer dans le tableau.

Exercice

Code :

```
#include <stdio.h>
#include <stdlib.h>

void main() {

        const int array_size = 5;
        int *array;
        array = (int *) malloc( array_size * sizeof( int) );

        int *p;

        // Initialization loop
        p = array;
        while (p < array + array_size) {
                *p = 42;
                p++;
        }

        printf("The array has been populated. We shall now display
it to the user (that is, you, dear reader :)\n"  );

        // Visualization loop
        p = array;
        while (p < array + array_size) {
                printf("%d\n", *p);
                ++p;
        }

        printf( "The array has been correctly displayed
(hopefully).\n" );

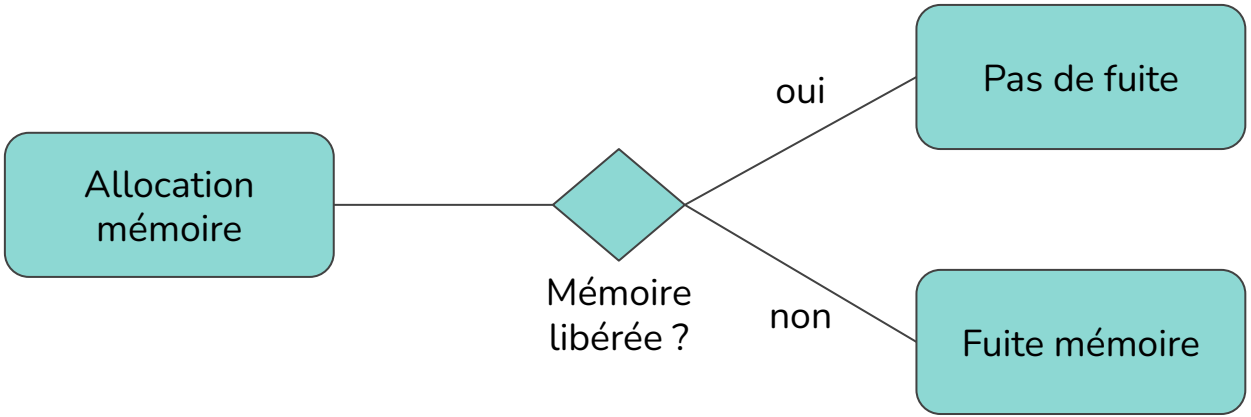
        // Due diligence
        free(array);
        array=NULL;
        p=NULL;
}
```

Fuites et Pointeurs Suspendus

Qu'est ce qu'une fuite mémoire ?

Définition

Mémoire qui, n'ayant pas été libérée correctement, n'est plus accessible mais reste allouée dans le programme.



L'amoncellement de fuites mémoires entraînent ainsi une consommation mémoire excessive au fil du temps.

En C, la plupart des fuites viennent d'une allocation dynamique qui n'est pas suivie d'un `free()`.

Les affres des fuites mémoires

Les fuites mémoire sont :

- souvent silencieuses. La quantité de mémoire perdue est souvent assez faible (du moins au début) pour ne pas être détectée sans test spécifique.
- Elles s'accumulent avec le temps. Le code touché étant bien souvent exécuté plus d'une fois, les fuites tendent à s'aggraver petit à petit.
- Au fur et à mesure que la mémoire est remplie, les performances du programme et même celles système se dégradent.
- Le programme finit soit par être tué par l'OS, soit par le faire planter.

Comment les éviter (ou les corriger)

Comment éviter les fuites mémoires ? C'est simple...

- Toujours associer une allocation dynamique (i.e. `malloc()`) à un `free()`.
- Validation systématique de l'allocation (`if (ptr == NULL)`).
- Retour du pointeur à `NULL` après utilisation.

Mais également :

- Implémenter des stratégies systématisant la bonne gestion de la mémoire (i.e. **réfléchir** à une stratégie mémoire)
- Utiliser des outils de débogage mémoire (Valgrind)

```
ptr = (…) malloc(…);
if (ptr == NULL) {
    // Erreur
    …
}
…
free(ptr);
ptr = NULL;
```

Déréférencement de pointeur NULL

Déréférencement sur un pointeur nul (NULL Pointer) : Cause l'arrêt du programme (Denial of Service/DoS).

- *Règle de l'ANSSI :* Ne jamais déréférencer un pointeur NULL.

L'utilisation de l'opérateur de déréférence sur un pointeur nul peut entraîner une erreur d'exécution ou un plantage du programme. Il est **très important** de s'assurer que votre pointeur pointe vers un emplacement mémoire valide avant de le déréférencer.

L'utilisation de l'opérateur de déréférence pose-t-elle des problèmes de sécurité ?

Oui, l'utilisation de l'opérateur de déréférence pose des problèmes de sécurité. Si vous déréférencez par erreur un pointeur non initialisé ou non valide, cela peut entraîner un comportement indéfini, conduisant à des plantages de programme, à une corruption de la mémoire ou à d'autres problèmes inattendus. Il est essentiel de manipuler les pointeurs avec précaution et de s'assurer de leur validité avant de les déréférencer.

Nettoyez derrière vous

Un **Pointeur Suspendu** (*Dangling Pointer*) est un pointeur qui continue de référencer une zone mémoire qui a été libérée par un `free()`.

En C, quand on libère de la mémoire allouée dynamiquement avec `free()`, l'adresse reste stockée dans le pointeur mais cette mémoire n'appartient plus au programme.

Tenter de l'utiliser à nouveau (en lecture ou en écriture) donne lieu à une erreur dite de "**Use-After-Free**" (UAF).

Une autre erreur courante est le **Double Free** : elle se produit quand on libérer la même mémoire deux fois, corrompant ainsi l'allocateur de mémoire.

Mitigations :

- Affecter le pointeur à NULL après désallocation.
- Vérifier la validité d'un pointeur avant usage.

```
int *p = malloc(sizeof(int));
*p = 42;
free(p);          // La mémoire est libérée
printf("%d\n", *p); // ❌ Use-After-Free : comportement indéfini
free(p); // ❌ Double free : corruption de la mémoire
```

Segfault 😔

SIGSEG, Segfault, Segmentation Fault, Segmentation Error, etc. Autant de termes pour transcrire une même réalité : votre programme plante parce que vous avez tenté d'accéder à un bloc mémoire que vous n'aviez pas le droit de toucher. C'est le problème le plus commun en C, surtout quand on débute.

Plus formellement, une erreur de segmentation se produit lorsqu'un programme C tente d'accéder à une zone mémoire non autorisée — c'est-à-dire une adresse qui ne lui appartient pas, ou qu'il n'a pas le droit de lire/écrire.

```
int *p = NULL;

*p = 5;   // ❌ Erreur de segmentation (écriture via un pointeur NULL)
```

Segfault 😔

Pourquoi est-ce possible ? Parce qu'en C, le compilateur ne vérifie pas automatiquement les accès mémoire : c'est donc au programmeur de garantir que chaque pointeur est valide avant utilisation.

Dans les faits, il s'agit souvent d'une conséquence critique d'une mauvaise gestion des pointeurs, notamment du déréférencement invalide de la mémoire :

Utilisation de pointeurs non initialisés

```
int *p;     // pointeur indéfini
*p = 10;    // ❌ accès à une adresse aléatoire
```

Déréférencement d'un pointeur NULL

```
int *p = NULL;
printf("%d", *p);    // ❌ accès à une adresse inexistante
```

Pointeur vers une variable locale hors de sa portée

```
int * ma_super_fonction() {
   int x = 42;
   return &x;  // ❌ accès hors du tableau
}
```

Dépassement de tableau (*out-of-bounds access*)

```
int tab[3] = {1,2,3};
printf("%d", tab[5]);    // ❌ accès hors du tableau
```

Utilisation d'un pointeur suspendu

```
int *p = malloc(sizeof(int));
free(p); *p = 10;     // ❌ use-after-free
```

Segfault 😔

Comment s'en prémunir ?

1. Initialiser tous les pointeurs (à NULL par exemple si on en a pas encore l'utilité)
2. Vérifier la validité d'un pointeur avant déréférencement
3. S'assurer que l'on respecte toujours les limites des tableaux
4. Utiliser des outils de débogage mémoire (valgrind par ex.)
5. Limiter l’usage de pointeurs globaux ou partagés : plus la portée est courte, plus le risque d’erreur diminue.
6. Toujours vérifier les retours des allocations dynamiques

Vulnérabilités critiques

Oh non, ça tourne en boucle (indéfiniment) !

Une **boucle infinie** (*infinite loop*) est une boucle dont la condition d'arrêt n'est jamais remplie.

Le programme reste donc bloqué, condamné à tourner en boucle ad vitam aeternam dans cette boucle et ne progresse plus :

```
int i = 0;
while (i < 10) {
    printf("%d\n", i);
    // ❌ i n'est jamais incrémenté → on a une boucle infinie
}
```

Causes fréquentes :

- Condition d'arrêt mal formulée

```
for (int i = 0; i >= 0; i++) {} // ❌ i ne devient jamais négatif
```

- Variable de condition jamais mise à jour

```
while (x < 100) {} // ❌ oubli d'incrémenter x
```

- Erreur logique dans la condition

```
while (p != NULL && p->next != NULL) { p = p->next; }
// ❌ p ne change jamais si la liste est vide
```

Oh non, ça tourne en boucle (indéfiniment) !

- Problème de flottants

```
float x = 0.0;
while (x != 1.0) { x += 0.1; }  // ❌ x n'atteint jamais exactement 1.0
```

- Attente d'un événement externe qui ne vient jamais

```
while (!data_ready) {
   // ❌ bloqué si data_ready reste faux
}
```

Comment s'en prémunir ?

1. Toujours vérifier la condition d'arrêt
2. Ajouter un mécanisme de sécurité (timeout ou compteur maximal)
3. Surveiller les entrées utilisateur
4. Utiliser des signaux ou drapeaux d'interruption
5. Débogage et instrumentation
6. Conception défensive

Débordement de buffer

Le **débordement de buffer** (*buffer overflow*) est un problème de sécurité fréquent dans de nombreux langages, dont C.

C'est une erreur **très courante**, souvent à l'origine de failles critiques (exécutions de code arbitraire, corruption mémoire, etc.).

Il survient lorsqu'un programme écrit **plus de données que la capacité prévue d'un tableau ou buffer** — typiquement un tableau de caractères (`char[]`) — provoquant une écriture hors limites de la mémoire.

Cependant C **ne vérifie pas automatiquement les bornes des tableaux** (rappel : c'est le travail du programmeur).

Débordement de buffer

Conséquences d'un débordement de buffer

- **Corruption de données** dans la mémoire voisine du buffer.
- **Plantage (*par une segfault*)** si on écrase des zones critiques.
- **Altération du flot d'exécution** (p. ex. retour de fonction, pointeurs) → **faille de sécurité majeure.**
- **Exécution de code arbitraire :** un attaquant peut injecter du code malveillant dans la pile (*stack smashing*).
- **Instabilité et comportement indéfini :** résultats aléatoires, erreurs intermittentes.

Débordement de buffer

Quelques exemples de débordement :

```
char texte[20];
scanf("%s", texte);  // ❌ pas de limite : peut écrire au-delà de 20 octets (20*sizeof(char))
```

```
char texte[10];
strcpy(texte, "Mon super ami !");  // ❌ débordement (15 caractères = 15 octets au lieu de 10)
```

```
char buffer[5];
for (int i = 0; i <= 5; i++){ buffer[] = 'A'; } // ❌ i=5 dépasse la taille du tableau
```

Débordement de buffer

Soit dit en passant, il y a d'autres options utiles (cf. doc GCC) :

```
-D_GLIBCXX_ASSERTIONS,
-ftrivial-auto-var-init=zero
-fPIE -pie -Wl, -z, -relro, -z,now
-fstack-protector-strong
-fstack-clash-protection
-fcf-protection=full
```

Comment limiter le problème ?

Toujours contrôler la taille des buffers :

- Allouer la bonne quantité de mémoire (autant que faire se peut).
- Ne pas oublier l'octet compris pour le `'\0'` de fin de chaîne.

Ne pas utiliser d'API ou de fonction dangereuse.

- Notamment, éviter les outils qui n'encadrent pas l'accès aux buffers.

Compiler avec les flags de protection :

- `-fhardened`, `-fstack-protector`, `-D_FORTIFY_SOURCE=2` (ou 3), `-Wall`, `-Wextra`

API Dangereuses (rappel)

`#include<string.h>`: `strcat()`, `strcpy()` car pas de vérification sur la taille des buffers

`strncat()`, `strncpy()` : où le 'n' représente la taille du buffer. Difficiles à utiliser.

`strlcpy()` et `strlcat()` : plus faciles à utiliser. 'l' □ tronquage des chaînes trop longues et garantit que le résultat est toujours une chaîne valide.

`fgets()` au lieu de `gets()` (qui est dangereuse et obsolète) et au lieu de `scanf()` qui a les mêmes problèmes que `strcat()` et `strcpy()`.

Sous Windows : utiliser `strcpy_s()` & `strcat_s()`.

Outils de débogage

GDB

GDB : pour analyser le plantage exact

Utilisation :

- Utiliser le flag -g en compilant avec GDB
- Puis lancer le programme dans gdb : gdb ./prog [`--args executablename arg1 arg2 arg3 `].
- Enfin, naviguer dans le programme selon votre besoin en utilisant les commandes ci-dessous (q pour quitter).

Commandes les plus importantes :

b N	point d'arrêt (breakpoint) à la ligne **N**
info break	liste les points d'arrêt
r	exécute le programme
c	continue jusqu'au prochain arrêt
p var	affiche la valeur de variable
s N	exécute les N prochaines instructions

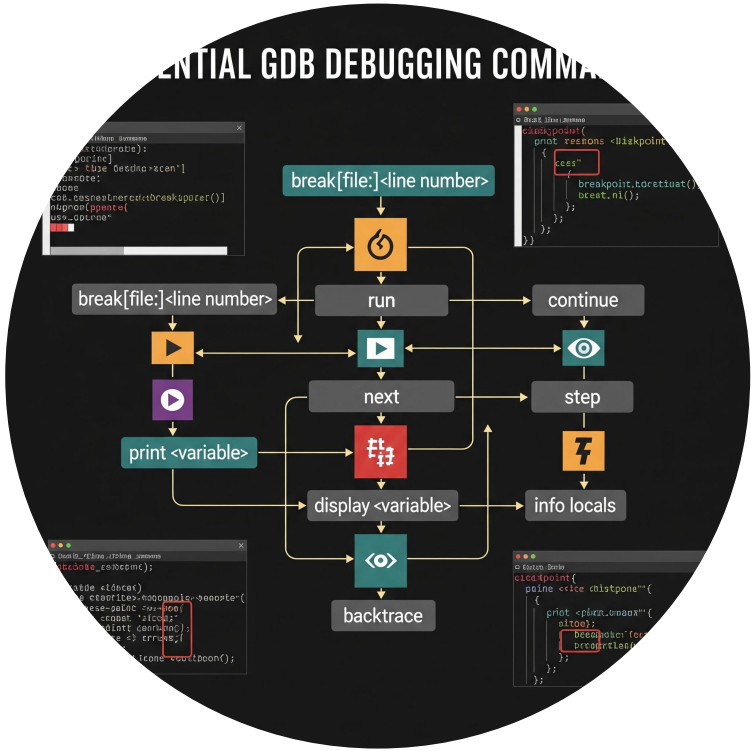
```c
int dichotomie(int v, int *tab, int
size) {
    int i, j;
    int mid;
    i = 0;
    j = size;
    while (i < j) {
        mid = (i + j) / 2;
        if (tab[mid] == v)
            return 1;
        if (tab[mid] < v)
            i = mid;
        else
            j = mid;
    }
    return 0;
}
```

```
$ gdb ./mon_programme
GNU gdb (GDB) 13.2
[bla bla bla de gdb...]
Reading symbols from
./mon_programme...

(gdb) run
```

cf. Valgrind et GDB

Commandes GDB



Pour info 👉

Une "trace de la pile" (stack trace) est une liste des appels de fonctions actifs à un moment donné de l'exécution d'un programme. Elle montre la séquence des fonctions qui ont été appelées pour arriver au point actuel du programme. En C, elle est souvent utilisée lors du débogage pour comprendre l'état du programme et identifier la cause d'un plantage ou d'une erreur.

Suite de la liste :

b main	point d'arrêt au début de la fonction main
b	point d'arrêt à la ligne actuelle
d N	supprimer le point d'arrêt à la ligne N
f	continue jusqu'à ce qu'à la sortie de la fonction actuelle
s	exécute la prochaine instruction
n	Comme s mais sans entrer dans les fonctions
bt	afficher la trace de la pile (*stack trace*) *ie.* là où l'on est dans le programme avec les couches de fonctions appelantes successives.
u	remonter d'un niveau dans la pile
d	descendre d'un niveau dans la pile
q	fermer GDB

Valgrind

Valgrind est un outil qui se comporte comme une machine virtuelle, émulant l'exécution de votre programme et mesurant l'utilisation de la mémoire par ce dernier.

Son but est de vous aider à identifier les problèmes de mémoire dans votre programme (utilisations incorrectes ou abusives comme les fuites mémoire ou les lectures/écritures hors limites de buffer).

Exemple où l'on détecte un accès hors limites à un tableau ➡

cf. Valgrind et GDB

1.

```c
#include <stdlib.h>

int main(void) {
    int *tableau = malloc(sizeof(int)
* 10);

    for (int i = 0; i <= 10; i++)
        tableau[i] = i;

    free(tableau);

    return 0;
}
```

2.

```
gcc -Wall -std=c17 -g exemple.c -o exemple
```

3.

```
$ valgrind ./exemple
==27466== Memcheck, a memory error detector
==27466== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==27466== Using Valgrind-3.21.0 and LibVEX; rerun with -h for copyright info
==27466== Command: ./exemple
==27466==
==27466== Invalid write of size 4
==27466==    at 0x10917F: main (main.c:7)
==27466==  Address 0x4a57068 is 0 bytes after a block of size 40 alloc'd
==27466==    at 0x4842794: malloc (in
/usr/libexec/valgrind/vgpreload memcheck-amd64-linux.so)
==27466==    by 0x10915A: main (main.c:4)
==27466==
==27466==
==27466== HEAP SUMMARY:
==27466==     in use at exit: 0 bytes in 0 blocks
==27466==   total heap usage: 1 allocs, 1 frees, 40 bytes allocated
==27466==
==27466== All heap blocks were freed -- no leaks are possible
==27466==
==27466== For lists of detected and suppressed errors, rerun with: -s
==27466== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

AddressSanitizer

AddressSanitizer détecte les erreurs de mémoire courantes en C et en C++ :

- Identifie les dépassements de tampon et "use-after-free"
- Intégré au compilateur pour une analyse rapide à partir de GCC 4.8 : utiliser le flag -fsanitize=address

Cf. HOWTO: Use Address Sanitizer | Ohio Supercomputer Center

1.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, const char *argv[]) {
    char *s = malloc(100);
    strcpy(s, "Hello world!");
    printf("string is: %s\n", s);
    return 0;
}
```

2.

```
gcc leak.c -o leak -fsanitize=address -static-libasan
```

3.

```
string is: Hello world!

=================================================================
==235624==ERROR: LeakSanitizer: detected memory leaks

Direct leak of 100 byte(s) in 1 object(s) allocated
from:
    #0 0x4eaaa8 in   interceptor malloc
../../../../libsanitizer/asan/asan_malloc_linux.cc:144
    #1 0x5283dd in main
/users/PZS0710/edanish/test/asan/leak.c:6
    #2 0x2b0c29909544 in   libc_start_main
(/lib64/libc.so.6+0x22544)

SUMMARY: AddressSanitizer: 100 byte(s) leaked in 1
allocation(s).
```

🔧 My LLM is my mechanic

OWASP Top 10 for LLM Applications 2025

↗

L'aide des LLM pour le débogage est prometteuse mais cela doit rester une **aide** (cf. LLM02 et LLM09 dans le rapport de l'OWASP sur les risques de sécurités des LLM et les notions de sur-dépendance et de confiance excessive).

➥ Validez toujours les suggestions du LLM.

➥ Ne partagez jamais d'informations sensibles avec un LLM (cela inclut bien entendu le code source propriétaire).

Comment débogger [1] ?

- Empiriquement, il semble qu'au bout de 2-3 itérations il est préférable de recommencer la conversation de déboggage à zéro.
- Utiliser des modèles spécialisés dans le développement de code (ou mieux affinés pour cet objectif).

Soit dit en passant, voici quelques lectures sur le sujet de la génération de code par IA : AI-Generated Code & Security, Superagency in the workplace: Empowering people to unlock AI's full potential (mais attention aux prises de position).

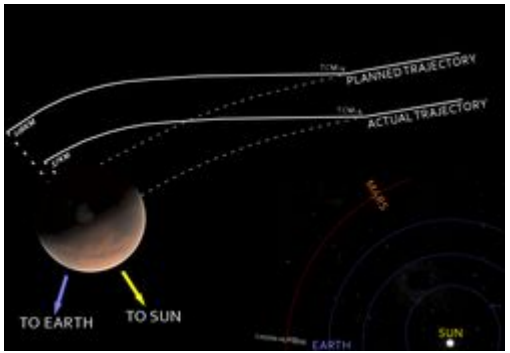
[1] Adnan, Muntasir, and Carlos CN Kuhn. "The Debugging Decay Index: Rethinking Debugging Strategies for Code LLMs." *arXiv preprint arXiv:2506.18403* (2025).

Conclusion

Principes fondamentaux de la programmation défensive en C



Programmation défensive… de quoi s'agit t-il ?





En une phrase : "faire confiance à l'utilisateur, aux autres développeurs, à un LLM ou même à soi-même, ou bien encore se dire que quelque chose ne peut pas arriver, c'est très dangereux !"





(OpenSSL Heartbleed)

Programmation défensive… de quoi s'agit t-il ?

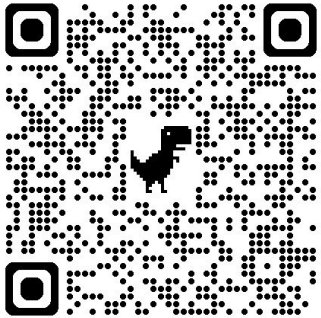
Définition

La programmation défensive est une approche de **conception** qui vise à rendre un logiciel **robuste** et **capable de fonctionner même dans des conditions imprévues**. L'objectif principal est de rendre le code plus fiable en **anticipant** activement toute erreur potentielle.

Attention cependant : programmation défensive ≠ paranoïa.

N'hésitez pas à refaire le quiz introductif→

(par contre, cette fois-ci vous avez le devoir d'obtenir un 10/10 ; sinon revoyez votre cours).



Principes clés

1. Ne jamais faire confiance aux données externes

Validez systématiquement toutes les entrées provenant de l'utilisateur, de fichiers, de réseaux, etc.

2. Échouer de manière sûre

Si une erreur se produit, le programme doit entrer dans un état sûr plutôt que de continuer avec des données corrompues.

3. Le code doit être simple et lisible

Un code complexe est un terrain fertile pour les bugs. La clarté (du code mais aussi de l'architecture du programme) est une forme de défense.

4. Utiliser des assertions pour valider les hypothèses

Documentez et vérifiez les préconditions et postconditions de toutes vos fonctions.

Exercice : Écrire une fonction `get_age()` qui demande l'âge de l'utilisateur. La fonction doit garantir que l'entrée est bien un nombre et qu'il est dans une plage valide (par exemple, 1 à 120). Si l'entrée est invalide, elle doit re-demander l'âge jusqu'à obtenir une valeur correcte.

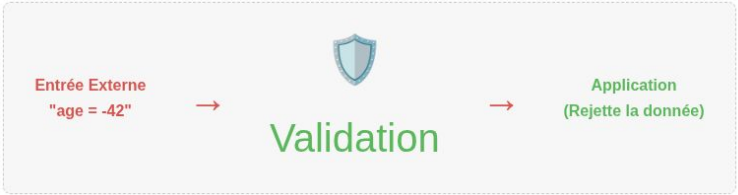
Le traitement des données d'entrée

Définition

Partir du principe que les **données** que vous recevez peuvent être **inattendues**, **imprévisibles** ou même **malveillantes**.

```
// Ne jamais faire confiance à l'entrée !
if (user_age < 0 || user_age > 120) {
    // Gérer l'erreur : c'est une valeur
                invalide.
        return ERROR_INVALID_AGE;
 }
 // Ici, on peut utiliser user_age en
 toute sécurité.
```

Toujours contrôler les données d'entrée de vos modules pour vous assurer que le code n'exécute que des choses qu'il maîtrise.



Exercice : Créer une fonction `divide(int a, int b, int* result)` qui calcule `a / b`. La fonction doit stocker le résultat dans le pointeur result. Elle doit gérer le cas de la division par zéro en renvoyant un code d'erreur (par exemple, '0' pour succès, '-1' pour erreur) sans faire planter le programme.

Gestion des erreurs

Chaque fonction susceptible d'échouer (allocation mémoire, opération sur fichier, etc.) **doit signaler cet échec** à l'appelant.

L'appelant, à son tour, doit vérifier **et réagir** à cette erreur. Ignorer (**ou pire, cacher**) une erreur peut mener à un comportement indéfini, à un crash ou être une porte ouverte aux détournements.

```c
void read file(const char* path) {
    FILE* f = fopen(path, "r");
    // Q : Que se passe-t-il si f est NULL
?
    // R : Le programme va planter ici :
    fgetc(f);
    fclose(f);
}
```



```c
#include <errno.h>
#include <string.h>

// Renvoie 0 en cas de succès, -1 en cas d'erreur
int read file(const char* path) {
    FILE* f = fopen(path, "r");
    if (f == NULL) {
        // perror affiche notre message + la description de
l'erreur système
        perror("Impossible d'ouvrir le fichier");
        return -1;
    }
    // ... traitement du fichier ...
    fclose(f);
    return 0;
}
```

Exercice : Corriger une fonction qui concatène deux chaînes de caractères de manière non sécurisée. La fonction `concat_strings(char* s1, char* s2)` doit allouer un nouveau buffer, y copier `s1` puis `s2`, et renvoyer le pointeur. Assurez-vous que l'allocation mémoire est vérifiée et que la taille est correcte.

Sécurité de la mémoire

Les erreurs de gestion de la mémoire sont **parmi les plus dangereuses** en C. Elles incluent :

- les dépassements de tampon (*buffer overflows*),
- l'utilisation après libération (*use-after-free*, *NULL pointer*),
- les **doubles libérations** (*double free*)
- et les **fuites mémoire** (*memory leaks*).

```c
// `gets` est extrêmement dangereux car il ne vérifie
// jamais la taille du buffer de destination.
void ask_name() {
    char name[32];
    printf("Entrez votre nom: ");
    gets(name); // Ne JAMAIS utiliser gets()
}
```



```c
// `fgets` est sûr car il prend la taille du buffer
// en argument, empêchant tout dépassement.
void ask_name() {
    char name[32];
    printf("Entrez votre nom: ");
    if (fgets(name, sizeof(name), stdin) != NULL) {
        // Traitement du nom...
    }
}
```

Exercice : Écrire une fonction `safe_add(int a, int b)` qui retourne la somme de `a` et `b`. Si l'addition risque de provoquer un dépassement de capacité positif (overflow), la fonction doit détecter le problème et, par exemple, afficher une erreur et terminer le programme (ou retourner une valeur spéciale, mais pour l'exercice, un `exit` est acceptable).

Sécurité des entiers

Nous n'avons pas tout évoqué dans ce cours (loin de là), mais voici 2 derniers éléments pour la route :

Les **dépassements d'entiers** (*integer overflows*) sont un bug (et une vulnérabilité) fréquent(e), en particulier lorsqu'ils sont utilisés pour calculer des tailles de buffer ou des indices de tableau (risque élevé d'accéder à une zone de mémoire invalide).

Un entier signé qui dépasse peut "revenir en arrière" (*wrap around*) et devenir négatif, tandis qu'un entier non signé revient à zéro.

```
// Si count est très grand, count * sizeof(int)
// peut causer un overflow. malloc recevra une
// petite taille, menant à un heap overflow plus
tard.
void process_items(int count) {
    int* items = malloc(count * sizeof(int));
    if (items) {
        // ...
    }
}
```



```
#include <limits.h>

void process_items(size_t count) {
    // Vérification avant la multiplication
    if (count > SIZE_MAX / sizeof(int)) {
        fprintf(stderr, "Erreur: Dépassement d'entier
potentiel.\n");
        return 1;
    }
    int* items = malloc(count * sizeof(int));
    if (items) {
        // ...
    }
}
```

Exercice : Ajouter des assertions pertinentes à une fonction `get array element(int* array, int size, int index)` qui retourne un élément d'un tableau. La fonction part du principe qu'elle ne sera jamais appelée avec des arguments incorrects.

Utiliser des assertions pour développer

Les **assertions** (#include <assert.h>) sont des outils très utiles lors du développement. Elles permettent de vérifier des conditions qui doivent TOUJOURS être vraies à un certain point du programme. Si la condition est fausse, le programme s'arrête immédiatement en indiquant où l'assertion a échoué.

C'est une excellente façon de documenter et de valider les préconditions (ce qui doit être vrai au début d'une fonction) et les postconditions (ce qui doit être vrai à la fin).

```c
#include <assert.h>

// Calcule la racine carrée, ne fonctionne que pour x >= 0
double calculate_sqrt(double x) {
    // Précondition: x ne doit pas être négatif.
    // Si cette condition est fausse, c'est un bug dans
    // le code qui appelle cette fonction.
    assert(x >= 0.0);

    // ... calcul ...
    return sqrt(x);
}
```

Cependant : les assertions sont généralement désactivées dans les builds de production (en définissant la macro `NDEBUG`). Elles ne doivent donc pas être utilisées pour la gestion d'erreurs d'exécution (comme valider une entrée utilisateur), mais pour détecter des bugs de programmation.

Une erreur est une erreur et doit être traitée comme telle (donc pas comme une assertion).

Cf. la discussion suivante : https://groups.google.com/g/comp.lang.c++.moderated/c/jgI11YyGNcI?pli=1